Advanced Visual Computing

Lab 3: point clouds processing and registration

Andrea Giachetti University of Verona, Italy
andrea.giachetti@univr.it

Open3D

- Powerful library for 3d data processing
  - Can handle point clouds (another potential tool is pcl)
- Let's use in the python codes in the Lab3 archive
- Documentation, tutorials and examples:
  - https://www.open3d.org/docs/latest/
- We will focus on the registration pipelines
  - https://www.open3d.org/docs/latest/tutorial/pipelines/index.html

Depth images

- Just to see how depth images are encoded
  - introduction_rgbd shows a file captured from depth camera and the basic processing to visualize point clouds using Open3d
- The RGB-D image is acquired with a PrimeSenseCarmine camera (old device)
  - The focal length is 525 for both axes (fx=525, fy=525)and the principal point is (cx =319.5,cy=239.5)
  - Rgb and d are obviously already registered (but not all depths are valid)
  - We can extract a mesh from the point cloud as we have done in the stereo code (see example codes)

Point cloud registration

- Remember:
  - Rigid registration 6DOF transform (translation+rotation)
    - Optimization problem to be solved given points correspondence
  - Typical approach for pre-registered clouds (local registration): ICP
    - Several variants and distances
  - Typical approach to get approximate registration: feature matching and robust optimization
    - Different features and robust estimators
  - Color information can help

ICP

- Iterative registration of point clouds
  - Nearest neighbor paring
  - Orthogonal procrustes problem solution

$$\begin{array}{ll} \underset{\Omega}{\text{minimize}} & \|\Omega A - B\|_F \\ \text{subject to} & \Omega^T \Omega = I, \end{array}$$

  - equivalent to finding the nearest orthogonal matrix to a given matrix $M = BA^T$,
  - solving the **closest orthogonal approximation problem**

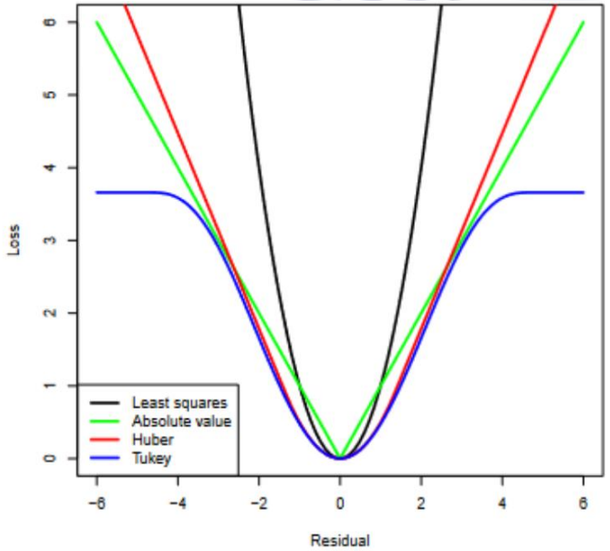
$$\min_R \|R - M\|_F \quad \text{subject to} \quad R^T R = I.$$

  - Solved with Singular Value Decomposition (SVD) $M = U\Sigma V^T$ that gives $R = UV^T$.
- We can write the code explicitly, without using open3D registration code, see
  - icp_ex.py

Open3D registration pipelines

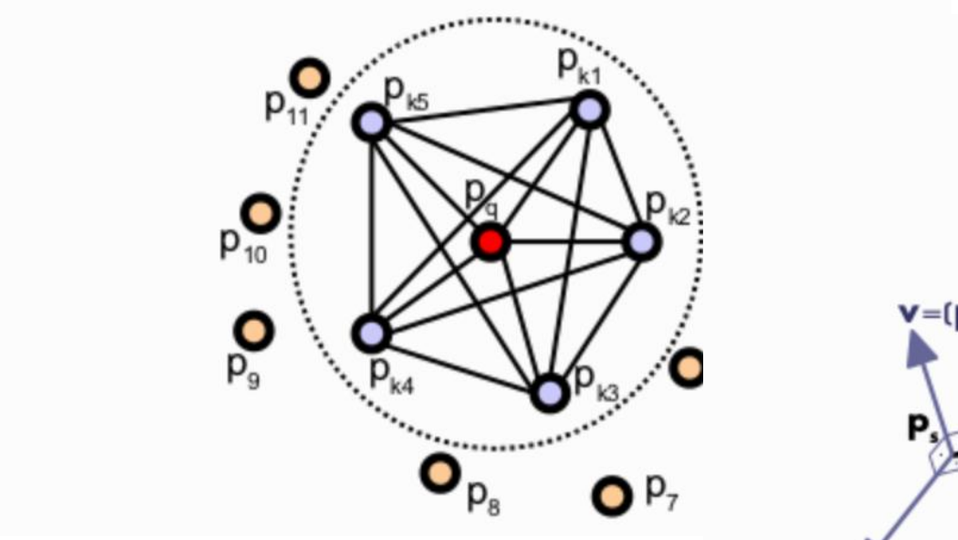
- See https://www.open3d.org/docs/release/tutorial/pipelines/index.html
  - ICP, comparing point to point and point to plane distances
    - icp_registration.py
  - Colored icp
    - colored_icp_registration.py
  - Robust ICP
    - robust_icp.py
- RANSAC for global registration
  - registration_ransac.py
    - https://www.open3d.org/docs/release/tutorial/pipelines/global_registration.html

Robust ICP: different losses



Global matching: feature detector/descriptor

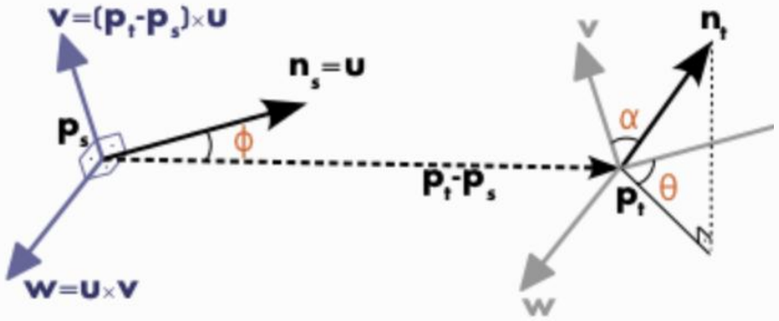
- FPFH (Fast Point Feature Histogram),
  - it attempts to capture as best as possible the sampled surface variations by taking into account all the interactions between the directions of the estimated normal
  - Point feature histogram: capture relationships between normals around the point



$$u = n_s$$

$$v = u \times \frac{(p_t - p_s)}{\|p_t - p_s\|_2}$$

$$w = u \times v$$



$$\alpha = v \cdot n_t$$

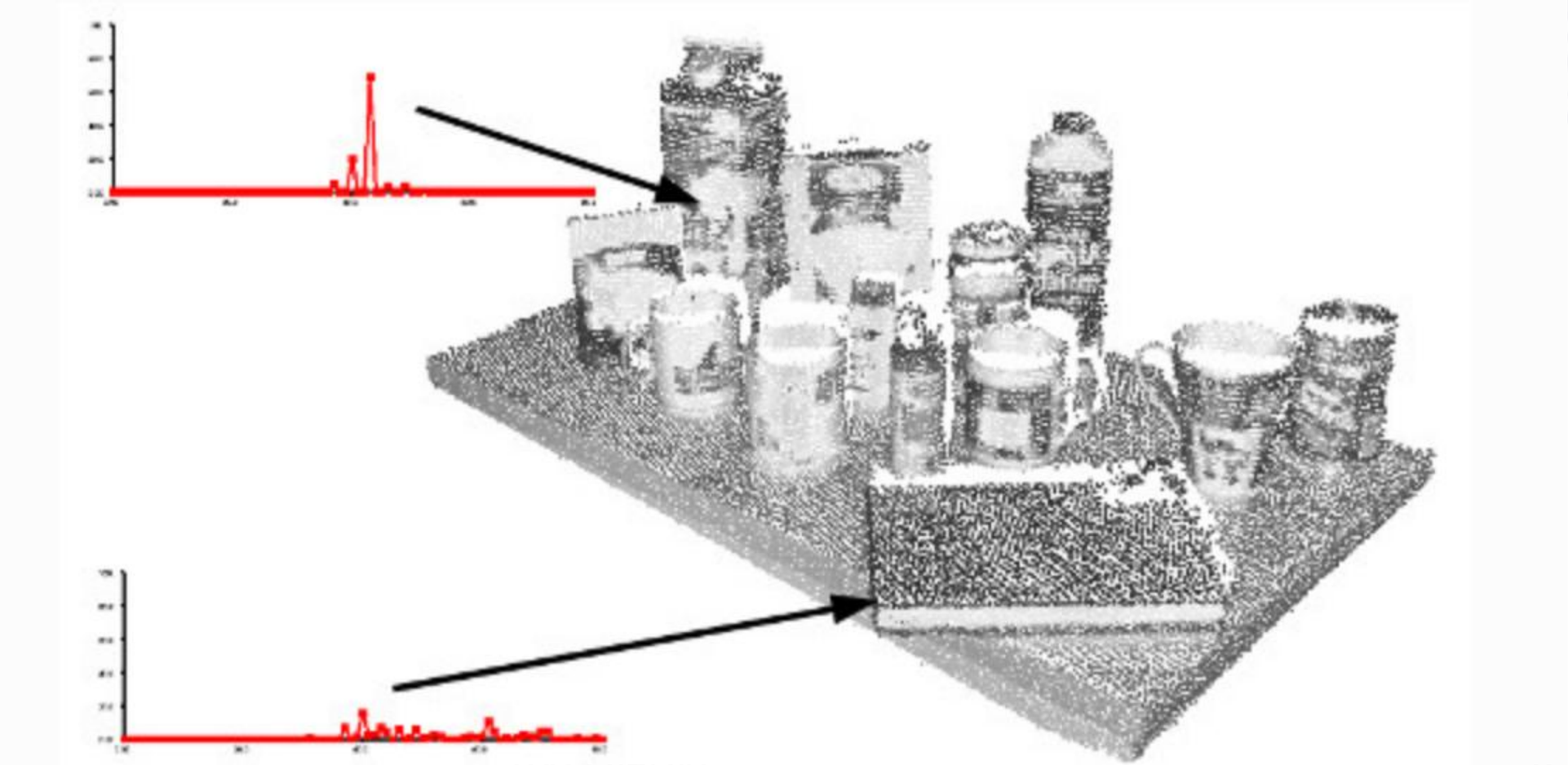
$$\phi = u \cdot \frac{(p_t - p_s)}{d}$$

$$\theta = \arctan(w \cdot n_t, u \cdot n_t)$$

From PCL https://pcl.readthedocs.io/

FPFH (Fast Point Feature Histogram),

- From PCL https://pcl.readthedocs.io/



Homework

- Consider the point clouds of the partial scans
  - Part1.ply
  - Part2.ply
- Using the functions seen in the tutorial, try to
  - Estimate with feature-based robust matching the transform and refine with ICP
  - Create a merged, registered point cloud
  - (optional: mesh using Poisson using meshlab)

Lessons plan

- No lesson on Tuesday, April 7
  - You can exploit the available room for working at the homework (or starting to work at the last homework, see next)
  - The course should end on May 14
    - And some of the 3 hours lessons will be reduced to 2 hours (possibly April 21/28, not sure, you will be informed)

Shape contest

- Before the end of the course (currently planned on May 12) students are invited to
  - Create a shape (triangle mesh) that will be manufactured during the last lesson of the course
    - Using vision based pipeline and post processing (meshing, smoothing etc.)
    - Adding designed parts (Using Blender)
    - Preparing it for printing (we'll see)
- The winning shape will be printed and given to the author